

Chapter 54: Enterprise Architecture

1. Overview

A single-team Angular app and a 40-team, 200-developer Angular platform are different engineering problems. At small scale, the compiler, the router, and a `shared/` folder are enough. At enterprise scale, the bottleneck stops being "can we write the code" and becomes "can 15 teams ship independently without breaking each other, without a 45-minute CI pipeline, without a shared `utils.ts` turning into a 4,000-line dumping ground, and without a design system that drifts into eleven slightly different buttons."

Enterprise Angular architecture is the discipline of putting *structural* guardrails around a codebase so that its social and organizational problems (who owns what, who can depend on whom, how fast can we test only what changed) are solved by tooling rather than by tribal knowledge and code review vigilance. The concrete levers are:

- **Domain-driven module/folder boundaries** — carving the app along business capabilities, not technical layers, so teams own vertical slices.
- **Nx monorepos** — apps/libs structure, tagging, computed project graphs, and `affected` commands that make CI scale sub-linearly with repo size.
- **Shared component libraries / design systems** — a single source of visual and interaction truth, versioned and consumed like any other dependency.
- **Feature flags** — decoupling deployment from release, enabling trunk-based development across teams that ship at different cadences.
- **Environment configuration strategy** — one build artifact promoted through dev → staging → prod, never "rebuild per environment."
- **Enforced coding standards** — ESLint custom rules, `dependency-cruiser`, and CI gates that make architecture violations a compile-time/lint-time failure instead of a post-mortem.
- **API layer abstraction** — insulating hundreds of components from backend contract churn behind a stable internal facade.

This chapter treats these as one connected system: the folder structure feeds the Nx tags, the tags feed the lint rules, the lint rules feed CI's affected-command scoping, and the whole thing exists to let many teams move in parallel without stepping on each other.

2. Core Concepts

2.1 Domain-Driven Boundaries: Why "Technical Layering" Fails at Scale

A classic small-app layout groups by *type*:

```
src/app/  
  components/  
  services/  
  models/  
  pipes/  
  directives/
```

This works until you have 30 people touching `services/`. Every PR touches the same folders, every merge conflicts, and nobody can answer "who owns this file" without `git blame` archaeology.

Domain-driven (vertical slice) layout groups by *business capability* instead:

```
src/app/  
  orders/  
    feature-order-list/  
    feature-order-detail/  
    data-access-orders/  
    ui-order-card/  
    util-order-formatting/  
  billing/  
    feature-invoices/  
    data-access-billing/  
    ui-invoice-table/  
  shared/  
    ui-design-system/  
    util-formatting/  
    data-access-auth/
```

Each domain (`orders`, `billing`) is a bounded context in the DDD sense — it has its own models, its own data-access layer, its own UI components, and ideally its own team. Cross-domain communication happens through well-defined public APIs (a domain's `index.ts` barrel), never by reaching into another domain's internals.

This is the folder-level expression of **Conway's Law**: system structure mirrors communication structure. If Team A owns `orders` and Team B owns `billing`, the codebase should make it physically awkward for Team B to import a private file three levels deep inside `orders/feature-order-detail/internal/`.

2.2 The Nx "Type × Scope" Library Taxonomy

Nx formalizes vertical slicing with a library classification scheme that combines a **type** and a **scope**:

Type	Purpose	Example
<code>feature</code>	Smart components, routed pages, orchestrates data-access + ui	<code>orders-feature-order-list</code>
<code>data-access</code>	State management, HTTP calls, NgRx/signals stores	<code>orders-data-access</code>
<code>ui</code>	Dumb/presentational components only, no business logic	<code>orders-ui-order-card</code>

Type	Purpose	Example
<code>util</code>	Pure functions, pipes, formatters, no Angular DI dependencies ideally	<code>orders-util-formatting</code>

Scope	Meaning
<code>orders</code> , <code>billing</code> , <code>shipping</code>	Domain-specific — only that domain's apps/libs may depend on it
<code>shared</code>	Cross-domain, stable, owned by a platform team

Combined as tags, e.g. `type:feature`, `scope:orders`. The rule of thumb dependency graph:

```
feature → data-access, ui, util
data-access → util
ui → util
util → (nothing app-specific)
```

Features can depend on anything below them; `util` should depend on almost nothing. This creates a **directed acyclic graph (DAG)** — the single most important invariant in a large Nx workspace, because Nx's affected-command performance and correctness both assume acyclicity.

2.3 Enforcing Boundaries: `@nx/enforce-module-boundaries`

Tags aren't just documentation — Nx ships an ESLint rule, `@nx/enforce-module-boundaries`, that reads the tags on each project and fails the lint step if an import violates the declared constraints. This turns "please don't import billing internals from orders" from a code-review nitpick into a build failure. Section 4 covers exactly how this is computed.

2.4 Shared Component Libraries and Design Systems

At enterprise scale, "shared UI" evolves through three maturity stages:

1. **Copy-paste convergence** — every team builds its own button, then someone notices five buttons and starts a `shared-ui` folder.
2. **Internal library** — the shared components move into an Nx lib (`ui-design-system`) or a separately versioned/published npm package, consumed by all apps.
3. **Design system with contract** — the library ships not just components but **design tokens** (colors, spacing, typography as CSS custom properties or SCSS variables generated from a single source, often Figma-exported JSON), a **Storybook** as the living contract/documentation, and **visual regression tests** (Chromatic, Percy) that fail CI when a change alters a component's rendered output unexpectedly.

Key architectural decisions for a design system library:

- **API surface stability:** presentational components should expose `@Input()` / `@Output()` (or signal inputs/outputs) contracts that change rarely and follow semver strictly, since dozens of consumers depend on them.
- **Theming:** CSS custom properties or Angular Material's theming API so consuming apps can reskin without forking the library.

- **Ownership:** usually a dedicated platform/DX team, with an RFC or contribution process for other teams adding components — otherwise the design system also devolves into a dumping ground.
- **Distribution:** either (a) an Nx lib inside the same monorepo, resolved via TypeScript path mapping, rebuilt on every relevant `affected` run — zero version skew, but requires everyone on one repo — or (b) a separately versioned/published npm package consumed across multiple repos, which introduces version-skew problems (see Edge Cases) but decouples release trains.

2.5 Nx Monorepo Architecture

Why a monorepo for enterprise Angular:

- Atomic cross-cutting changes: a shared library's breaking change and all its consumers' fixes land in one commit, one PR, one CI run — no "please bump to library v5 in your app" coordination across repos.
- Single source of truth for tooling versions (Angular version, TypeScript version, ESLint config) — no drift between 15 separate repos each frozen at a different Angular major.
- Code sharing without publishing overhead for internal-only libraries.

Structure:

```
myworkspace/
  apps/
    admin-portal/
    admin-portal-e2e/
    customer-app/
    customer-app-e2e/
  libs/
    orders/
      feature-order-list/
      data-access/
      ui-order-card/
    billing/
      feature-invoices/
      data-access/
    shared/
      ui-design-system/
      util-formatting/
      data-access-auth/
      feature-shell/
  nx.json
  tsconfig.base.json
  eslint.config.js (or .eslintrc.json)
```

Each `apps/*` is a deployable unit (its own `angular.json` / `project.json` build target, its own environment files). Everything reusable lives in `libs/*`. Apps should ideally be thin — a shell that imports routed `feature` libs and wires up bootstrapping, DI providers, and environment-specific config. This "thin app / fat libs" pattern maximizes what `affected` can skip (see 4.1) and makes it trivial to spin up a second app (e.g., a white-label build) that reuses 90% of the libs.

Tags in `project.json`:

```
{
  "name": "orders-data-access",
  "tags": ["type:data-access", "scope:orders"]
}
```

Affected commands (`nx affected -t test`, `nx affected -t lint`, `nx affected -t build`) compute, from a git diff, the minimal set of projects whose tests/lints/builds could possibly be impacted, and run tasks only for those — plus Nx's distributed task caching (local or via Nx Cloud) means even *unaffected-but-previously-built* tasks are replayed from cache rather than recomputed. This is what keeps CI on a 300-project workspace roughly as fast as CI on a 20-project workspace for a typical small PR.

2.6 Versioning and Publishing Internal Libraries

Two competing models:

- **Monorepo-internal, unversioned (path-mapped)**: libraries are consumed via TS path aliases (`@myorg/orders/data-access`), never published to a registry, always built from source at the version that's currently checked out. Every consumer is always on the latest version because there is only one version, in one repo, at any time. This is the default Nx model and avoids version-skew entirely, at the cost of requiring all consuming apps to live in the same repo.
- **Published packages (npm/verdaccio/private registry)**: used when libraries need to be consumed by teams/repos outside the monorepo, or for a public/semi-public design system. This requires:
 - **Semantic versioning** enforced by tooling like `nx release` (or Lerna/Changesets historically) — patch for fixes, minor for additive API, major for breaking changes.
 - **Changelogs generated from conventional commits** (`feat:`, `fix:`, `BREAKING CHANGE:`) so `nx release` can compute the correct version bump and changelog automatically.
 - **A deprecation policy**: breaking changes get a deprecation warning release before removal, giving consumers a migration window (often paired with `ng update` schematics that codemod the consumer's code automatically — the same mechanism Angular itself uses for its own breaking changes).

Most large orgs use a hybrid: fast-moving domain libs stay path-mapped inside the monorepo; the design system and a handful of truly cross-repo utilities get published with strict semver.

2.7 Feature Flags

Feature flags decouple **deployment** (code reaching production) from **release** (a user being able to see/use it). This enables:

- **Trunk-based development**: teams merge incomplete features to `main` behind a flag, avoiding long-lived feature branches and their merge hell.
- **Progressive rollout**: 1% → 10% → 100% of users, or specific customer segments (enterprise tier only, internal dogfood ring, specific geography).
- **Kill switches**: instantly disable a misbehaving feature without a redeploy.
- **A/B testing and experimentation**.

Flag categories that matter architecturally:

- **Release flags** — short-lived, removed once a feature is fully rolled out. These are the ones prone to

becoming debt (see 5.4).

- **Ops/kill-switch flags** — long-lived by design, protect against backend instability.
- **Permission/entitlement flags** — long-lived, tied to subscription tier, not really "flags" so much as authorization — often modeled separately from the flagging system to avoid confusion.

Architecturally, flags should be accessed through a single abstraction (a `FeatureFlagService`) rather than scattered `if (localStorage.getItem(...))` checks, so the *source* of flags (LaunchDarkly, a home-grown config service, environment JSON, ConfigCat) can change without touching consumer code — this is the same "abstract the volatile thing behind a stable interface" principle as the API layer in 2.8.

2.8 Environment Configuration Strategy

The cardinal rule: **build once, configure per environment at deploy time** — never rebuild the Angular bundle separately for dev/staging/prod. A rebuild-per-environment strategy means the artifact that passed QA in staging is *not* the artifact that ships to prod (different compilation), which defeats the purpose of staging as a gate.

Angular CLI's classic `environment.ts` / `environment.prod.ts` file-replacement mechanism is fine for build-time constants (like `production: true` toggling Angular's dev-mode checks) but is explicitly the wrong tool for anything that should differ *without* a rebuild — API base URLs, feature-flag service endpoints, client IDs — because file replacement bakes the value into the JS bundle at build time.

The enterprise-correct pattern is **runtime configuration**:

1. A `config.json` (or an endpoint) is fetched once during app initialization via an `APP_INITIALIZER` (or the newer `provideAppInitializer`), before the app renders.
2. That JSON is deployed alongside the static assets, and *only that file* differs between dev/staging/prod — the JS/CSS bundle is byte-identical across all three.
3. A typed `ConfigService` exposes the parsed values to the rest of the app, so components never touch `window` or raw JSON.

This gives you a single build promoted through environments (true CI/CD, provenance-preserving), while still allowing per-environment values, and it lets ops rotate an API endpoint without invoking the Angular build pipeline at all.

2.9 Enforced Coding Standards and Architecture-as-Code

At small scale, architecture is a wiki page nobody reads. At enterprise scale, it must be **encoded into tools that run in CI and fail the build**:

- **ESLint custom rules** — beyond `@nx/enforce-module-boundaries`, teams write custom ESLint rules (using `@typescript-eslint/utils` for AST-aware rules) for org-specific conventions: "no direct `HttpClient` injection outside `data-access` libs," "all public API exports must have JSDoc," "no `any` in `feature` libs," "component selectors must be prefixed `wm-`."
- **dependency-cruiser** — a general-purpose dependency-graph linter that works even outside Nx (plain Angular workspaces, or cross-checking things Nx's own rule doesn't cover, like disallowing imports of test files from non-test files, or forbidding a `core` module from importing anything from `features`). It reads a rules config (regex-based `from` / `to` matchers) and can output a fail/warn per violation plus a

visual graph (`.svg`) of the dependency structure — useful in architecture reviews to *see* whether the DAG has grown cycles.

- **CI gates:** PR pipelines run `nx affected -t lint, test, build, e2e` and treat the custom rules and `dependency-cruiser` output as blocking checks, not advisory ones. Architecture Decision Records (ADRs) document *why* a boundary exists; the lint rule is what actually *enforces* it.

2.10 API Layer Abstraction for Backend Contract Changes

In a large app, dozens of components indirectly depend on backend response shapes. If every component calls `HttpClient` directly and maps raw DTOs inline, a backend contract change (renamed field, restructured nested object, v1→v2 API migration) becomes a shotgun-surgery refactor across the whole codebase.

The fix is a strict layering discipline inside each domain's `data-access` lib:

```
Component (feature lib)
  ↓ depends on
Facade / Store (domain models, app-friendly shape)
  ↓ depends on
Repository / API service (owns HTTP calls, DTO ⇔ domain-model mapping)
  ↓ depends on
HttpClient
```

- The **API/Repository layer** is the *only* place that knows the wire format (DTOs) and does the mapping to internal **domain models**. If the backend renames `cust_id` to `customerId` or restructures a nested resource, exactly one mapping function changes.
- The **Facade/Store** exposes domain models and use-case-shaped methods (`getActiveOrders()` , not `getOrdersWhereStatusEquals('ACTIVE')` mirroring a query-param name) so components are shielded from query-shape churn too.
- **Versioned API clients:** when a backend introduces `/v2/orders` , the repository layer can support both DTO shapes behind one facade method, and the migration is invisible to every component — the facade picks the version via a flag or gradually cuts over.
- This is essentially the **Anti-Corruption Layer** pattern from DDD applied to a frontend: your domain model must not corrode into a mirror of whatever the backend team ships this sprint.

3. Code Examples

3.1 Nx Workspace Layout with Tags and Module-Boundary Lint Rules

```
myworkspace/
├─ apps/
│  ├─ customer-portal/
│  │  └─ src/
│  │     └─ app/app.config.ts
│  │     └─ main.ts
│  │     └─ assets/config/config.json
│  │     └─ project.json
│  └─ admin-portal/
│     └─ project.json
```

```

└─ libs/
  └─ orders/
    └─ feature-order-list/
      └─ project.json      # tags: [type:feature, scope:orders]
    └─ data-access/
      └─ project.json      # tags: [type:data-access, scope:orders]
    └─ ui-order-card/
      └─ project.json      # tags: [type:ui, scope:orders]
  └─ billing/
    └─ feature-invoices/
      └─ data-access/
        └─ shared/
          └─ ui-design-system/    # tags: [type:ui, scope:shared]
          └─ util-formatting/    # tags: [type:util, scope:shared]
          └─ data-access-auth/    # tags: [type:data-access, scope:shared]
          └─ feature-flags/      # tags: [type:data-access, scope:shared]
└─ nx.json
└─ tsconfig.base.json
└─ eslint.config.js

```

`libs/orders/data-access/project.json`:

```

{
  "name": "orders-data-access",
  "sourceRoot": "libs/orders/data-access/src",
  "projectType": "library",
  "tags": ["type:data-access", "scope:orders"],
  "targets": {
    "test": { "executor": "@nx/jest:jest" },
    "lint": { "executor": "@nx/eslint:lint" }
  }
}

```

`tsconfig.base.json` path mapping (how libs are imported without publishing):

```

{
  "compilerOptions": {
    "paths": {
      "@myorg/orders/feature-order-list": ["libs/orders/feature-order-list/src/index.ts"],
      "@myorg/orders/data-access": ["libs/orders/data-access/src/index.ts"],
      "@myorg/orders/ui-order-card": ["libs/orders/ui-order-card/src/index.ts"],
      "@myorg/shared/ui-design-system": ["libs/shared/ui-design-system/src/index.ts"],
      "@myorg/shared/util-formatting": ["libs/shared/util-formatting/src/index.ts"]
    }
  }
}

```

Root ESLint config enforcing the type/scope DAG with `@nx/enforce-module-boundaries`:

```

// eslint.config.js
const nx = require('@nx/eslint-plugin');

```



```

module.exports = [
  ...nx.configs['flat/base'],
  {
    files: ['**/*.ts'],
    rules: {
      '@nx/enforce-module-boundaries': [
        'error',
        {
          enforceBuildableLibDependency: true,
          allow: [],
          depConstraints: [
            // scope constraints: a domain lib may only depend on its own
            // domain or on shared – never on a sibling domain directly.
            {
              sourceTag: 'scope:orders',
              onlyDependOnLibsWithTags: ['scope:orders', 'scope:shared'],
            },
            {
              sourceTag: 'scope:billing',
              onlyDependOnLibsWithTags: ['scope:billing', 'scope:shared'],
            },
            {
              sourceTag: 'scope:shared',
              onlyDependOnLibsWithTags: ['scope:shared'],
            },
            // type constraints: enforce the feature -> data-access/ui -> util DAG
            {
              sourceTag: 'type:feature',
              onlyDependOnLibsWithTags: ['type:feature', 'type:data-access', 'type:ui',
'type:util'],
            },
            {
              sourceTag: 'type:data-access',
              onlyDependOnLibsWithTags: ['type:data-access', 'type:util'],
            },
            {
              sourceTag: 'type:ui',
              onlyDependOnLibsWithTags: ['type:ui', 'type:util'],
            },
            {
              sourceTag: 'type:util',
              onlyDependOnLibsWithTags: ['type:util'],
            },
          ],
        },
      ],
    },
  },
];

```

With this in place, `libs/billing/feature-invoices` importing anything from `libs/orders/data-access` fails lint immediately: `scope:billing` is not permitted to depend on `scope:orders`. Likewise, `orders-ui-order-card` (`type:ui`) importing `orders-data-access` (`type:data-access`) fails because `type:ui` may only depend on `type:ui / type:util`.

3.2 Feature Flag Service Pattern

```
// libs/shared/feature-flags/src/lib/feature-flag.service.ts
import { Injectable, signal, computed, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';

export interface FeatureFlagSet {
  [flagKey: string]: boolean | string | number;
}

/** Abstraction over whatever flag provider is behind the scenes
 * (LaunchDarkly, ConfigCat, a home-grown endpoint, static JSON).
 * Consumers never talk to the provider directly. */
export abstract class FeatureFlagProvider {
  abstract loadFlags(userContext: FlagUserContext): Promise<FeatureFlagSet>;
}

export interface FlagUserContext {
  userId: string;
  tenantId: string;
  rolloutGroup?: string;
}

@Injectable({ providedIn: 'root' })
export class HttpFeatureFlagProvider extends FeatureFlagProvider {
  private http = inject(HttpClient);

  override async loadFlags(ctx: FlagUserContext): Promise<FeatureFlagSet> {
    return firstValueFrom(
      this.http.post<FeatureFlagSet>('/api/flags/evaluate', ctx)
    );
  }
}

@Injectable({ providedIn: 'root' })
export class FeatureFlagService {
  private provider = inject(FeatureFlagProvider);

  private readonly _flags = signal<FeatureFlagSet>({});
  private readonly _ready = signal(false);

  readonly ready = this._ready.asReadonly();

  /** Call once during app init (see APP_INITIALIZER usage below). */
  async initialize(ctx: FlagUserContext): Promise<void> {
    try {
```

```

    const flags = await this.provider.loadFlags(ctx);
    this._flags.set(flags);
  } catch {
    // Fail-safe: fall back to all-off rather than blocking app startup.
    this._flags.set({});
  } finally {
    this._ready.set(true);
  }
}

isEnabled(key: string): boolean {
  return this._flags()[key] === true;
}

/** Reactive variant for use in templates / computed(). */
flag(key: string) {
  return computed(() => this._flags()[key] === true);
}

variant<T extends string>(key: string, fallback: T): T {
  const value = this._flags()[key];
  return (typeof value === 'string' ? (value as T) : fallback);
}
}

```

Wiring initialization so flags are resolved before the app renders (avoids UI flicker/flash-of-wrong-content):

```

// apps/customer-portal/src/app/app.config.ts
import { ApplicationConfig, inject, provideAppInitializer } from '@angular/core';
import { FeatureFlagService } from '@myorg/shared/feature-flags';
import { AuthService } from '@myorg/shared/data-access-auth';

export const appConfig: ApplicationConfig = {
  providers: [
    provideAppInitializer(async () => {
      const flags = inject(FeatureFlagService);
      const auth = inject(AuthService);
      const user = await auth.getCurrentUser();
      await flags.initialize({ userId: user.id, tenantId: user.tenantId });
    }),
  ],
};

```

Usage in a component — note the flag is consumed via the service abstraction, never via a raw config lookup:

```

@Component({
  selector: 'wm-order-list-page',
  template: `
    @if (newOrderFilterUi()) {
      <wm-order-filter-v2 />
    } @else {
      <wm-order-filter-legacy />
    }
  `
})

```

```

    }
  },
})
export class OrderListPageComponent {
  private flags = inject(FeatureFlagService);
  protected newOrderFilterUi = this.flags.flag('orders.filter-ui-v2');
}

```

3.3 Environment Configuration Abstraction (Runtime Config, Not Build-Time)

```

// libs/shared/util-config/src/lib/app-config.model.ts
export interface AppConfig {
  apiBaseUrl: string;
  featureFlagsEndpoint: string;
  authClientId: string;
  environmentName: 'local' | 'dev' | 'staging' | 'production';
  sentryDsn: string | null;
}

```

```

// libs/shared/util-config/src/lib/config.service.ts
import { Injectable, signal } from '@angular/core';
import { AppConfig } from '../app-config.model';

@Injectable({ providedIn: 'root' })
export class ConfigService {
  private _config = signal<AppConfig | null>(null);

  get value(): AppConfig {
    const cfg = this._config();
    if (!cfg) {
      throw new Error('ConfigService accessed before initialization - check APP_INITIALIZER wiring.');
```

```

    }
    return cfg;
  }

  async load(configUrl = '/assets/config/config.json'): Promise<void> {
    const res = await fetch(configUrl, { cache: 'no-store' });
    if (!res.ok) {
      throw new Error(`Failed to load runtime config from ${configUrl}: ${res.status}`);
    }
    const cfg = (await res.json()) as AppConfig;
    this._config.set(cfg);
  }
}

```

```
// apps/customer-portal/src/app/app.config.ts
import { provideAppInitializer, inject } from '@angular/core';
import { ConfigService } from '@myorg/shared/util-config';

export const appConfig: ApplicationConfig = {
  providers: [
    provideAppInitializer(() => inject(ConfigService).load()),
    // ... other providers
  ],
};
```

`apps/customer-portal/src/assets/config/config.json` — this is the **only** artifact that differs per environment; the compiled JS/CSS bundle is identical across dev/staging/prod:

```
{
  "apiBaseUrl": "https://api.staging.myorg.com",
  "featureFlagsEndpoint": "https://flags.staging.myorg.com",
  "authClientId": "staging-client-id",
  "environmentName": "staging",
  "sentryDsn": "https://staging-dsn@sentry.io/123"
}
```

An `HttpClient` interceptor (or the repository layer from 2.10) reads `ConfigService.value.apiBaseUrl` instead of a build-time constant, so switching environments is a config-file swap in the deploy pipeline, never a rebuild:

```
// libs/shared/data-access-http/src/lib/api-base-url.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { ConfigService } from '@myorg/shared/util-config';

export const apiBaseUrlInterceptor: HttpInterceptorFn = (req, next) => {
  const config = inject(ConfigService);
  if (req.url.startsWith('/api/')) {
    return next(req.clone({ url: `${config.value.apiBaseUrl}${req.url}` }));
  }
  return next(req);
};
```

4. Internal Working

4.1 How Nx Computes the Project Graph and `affected`

Nx maintains a **project graph**: a node per project (app or lib), edges representing dependencies. Edges come from two sources:

1. **Explicit static analysis** of import statements (TypeScript AST parsing) resolved through the `tsconfig` path mappings — if `orders-feature-order-list/src/index.ts` imports `@myorg/orders/data-access`, Nx adds an edge.
2. **Implicit dependencies** declared manually in `project.json` / `nx.json` for things static analysis can't see

(e.g., an app depends on a lib only referenced via a dynamic string in a config file).

This graph is cached (`.nx/cache` / the daemon process keeps it warm) and recomputed incrementally as files change, rather than from scratch on every command — Nx runs a background daemon that watches the filesystem and updates the graph reactively.

File hashing is the core primitive behind `affected`: Nx computes a content hash for every file in the workspace (and hashes of the resolved `tsconfig`, `nx.json`, lock file, and each project's configuration, since those also affect build output). For a given task (e.g., `test` for `orders-data-access`), Nx computes a **task hash** as a combination of:

- hashes of all source files in that project,
- hashes of all source files in every project it *transitively depends on* (per the project graph),
- the hash of the relevant executor/target configuration,
- the hash of relevant environment variables and CLI flags for that run.

`nx affected` specifically: Nx diffs the current git ref against a base (default: the base of the current branch, or explicitly `--base=main`), gets the list of changed files, maps each changed file to the project(s) that own it, then walks the project graph to find every project that transitively depends on any changed project (i.e., every ancestor in the dependency graph, since a change in a dependency can affect a dependent's behavior). That final project set is "affected," and only those projects' tasks run.

Distributed task caching: because the task hash is content-addressed and deterministic, if the *exact same hash* was computed on a previous run (locally, or by any teammate/CI agent sharing an Nx Cloud cache), Nx skips execution entirely and replays the cached terminal output plus restores cached output artifacts. This is why a monorepo with 300 libraries can still give a 3-file PR a sub-2-minute CI run: almost every task hash for unaffected projects matches a prior cache hit, so `affected` narrows the *candidate* set and caching skips re-execution for anything whose hash didn't actually change (including many "affected-by-graph-position-but-not-really-affected-by-content" cases).

This is precisely why the DAG (2.2) matters operationally, not just stylistically: a cycle between two libs means a change to either invalidates both, and worse, Nx's `affected`/topological-sort logic assumes acyclicity for correctly ordering task execution (a lib must be built before its dependents) — a genuine cycle either breaks task ordering or forces Nx to collapse the cycle into one unit, destroying the granularity `affected` relies on.

4.2 How ESLint Module-Boundary Rules Statically Enforce Architecture

`@nx/enforce-module-boundaries` is an ESLint rule, meaning it runs as part of the **static analysis** pass over each source file's AST, not at runtime and not via the bundler. Its mechanism:

1. For the file currently being linted, Nx determines which **project** owns it (by matching the file path against each project's `sourceRoot` / root in the project graph) and reads that project's `tags` array.
2. For every `import` / `export ... from` statement in the file, the rule resolves the imported module specifier to an actual project in the graph — using the same `tsconfig` path-mapping resolution the TypeScript compiler would use — and reads *that* project's tags.
3. It then checks the configured `depConstraints`: for each constraint whose `sourceTag` matches one of the current project's tags, the imported project must have at least one tag in that constraint's

`onlyDependOnLibsWithTags` list. If not, the rule reports a lint error at the import statement's location, naming the violating source and target tags.

4. Separately, the rule can also detect and forbid **circular dependencies** between projects (a project transitively importing back into itself through the graph) and can enforce that "buildable" libraries only depend on other buildable libraries (`enforceBuildableLibDependency`) so incremental/distributed builds stay valid.

Because this runs at lint time — which is itself scoped by `affected` in CI — an architecture violation is caught in seconds on the PR that introduces it, with a precise file-and-line error, rather than discovered later via a runtime stack trace, a bundle-size regression, or a "wait, why does billing know about orders internals" question in a design review six months later. The enforcement is purely static: it never executes any application code, so it's fast, deterministic, and can't be bypassed by conditional/dynamic imports that a human reviewer might miss but the resolved-specifier analysis still catches (dynamic `import()` expressions are analyzed the same way when statically resolvable).

5. Edge Cases & Gotchas

5.1 Circular Library Dependencies

Two libs importing from each other (directly, or transitively through a chain: $A \rightarrow B \rightarrow C \rightarrow A$) breaks the DAG assumption that `affected`'s change-propagation and any topological build ordering rely on. Symptoms: Nx either errors outright ("Circular dependency detected") when the boundary/cycle-detection rule is enabled, or, if undetected, causes over-broad "affected" sets (a trivial change anywhere in the cycle marks the entire cycle affected, forever, since nothing in a cycle is a strict "leaf"), and can cause incremental/buildable-library task graphs to deadlock or silently fall back to non-incremental full rebuilds.

Root cause is almost always a `util` or `shared` lib reaching back into a `feature / data-access` lib for "just one helper," or two domain libs that both want a piece of each other's model — which is usually a sign that shared piece belongs in a third, lower-level `shared`-scoped lib that both depend on downward, rather than living in either domain lib.

Fix: extract the shared piece into a new lib at a lower layer of the DAG (often `scope:shared`, `type:util`), and enable `@nx/enforce-module-boundaries`'s circular-dependency check so this class of bug fails CI immediately instead of being discovered by `affected` behaving strangely.

5.2 Over-Fragmentation into Too Many Tiny Libs

Nx's docs and community culture historically encouraged aggressive splitting ("one lib per component"), but past a certain point this backfires:

- **Cognitive overhead:** navigating 40 libs of 2 files each to understand one feature is worse than navigating one cohesive folder.
- **Barrel-file (`index.ts`) explosion:** every tiny lib needs its own public API surface, multiplying boilerplate and making "what's actually exported from this domain" harder to answer, not easier.
- **Build/test overhead per project:** even with caching, each project carries fixed overhead (project.json parsing, executor startup, Jest/webpack config resolution) that adds up when there are hundreds of near-empty libs.
- **False granularity:** splitting `ui-order-card` and `ui-order-card-header` into separate libs when they're never independently reused doesn't buy isolation, it just adds import indirection.

The corrective heuristic (matching later official Nx guidance): split libs at **actual reuse or actual team-ownership boundaries**, not preemptively. A folder-within-a-lib is fine until two different consumers need to depend on only part of it, or two different teams need independent ownership/versioning of it — that's the signal to extract a new lib, not "this component is getting big."

5.3 Shared Library Versioning Hell Across Teams

When the design system (or any shared lib) is a **published, versioned package** consumed by multiple repos (rather than a path-mapped in-monorepo lib), classic dependency hell returns:

- **Version skew:** App A is on `ui-design-system@4.2`, App B is stuck on `3.9` because upgrading requires absorbing an unrelated breaking change bundled into the same major bump. Users now see visually/behaviorally inconsistent components across the org's products.
- **Diamond dependency problems:** two shared libs (`ui-design-system` and `data-access-auth`) each depend on a third lib (`util-formatting`) at incompatible versions, and a consumer app ends up with two copies bundled, or a resolution conflict.
- **Breaking-change coordination cost:** a genuinely necessary breaking change (accessibility fix that changes a required `@Input`) now needs a deprecation cycle, a codemod/schematic, and multi-sprint coordination across every consuming team, instead of a single-commit fix that ripples through `affected` in a monorepo.

Mitigations: strict semver discipline plus automated changelogs (`nx release`, Changelsets); `ng update`-style migration schematics shipped alongside breaking changes so consumers get an automated codemod instead of manual find-and-replace; a deprecation window (mark-deprecated-then-remove-next-major) enforced by policy; and, where organizationally possible, preferring the in-monorepo path-mapped model precisely to sidestep this class of problem — version skew is a monorepo's biggest structural advantage over polyrepo for shared UI.

5.4 Feature Flag Debt Accumulation

Flags are cheap to add and easy to forget to remove. Left unchecked:

- **Combinatorial explosion of code paths:** N flags means up to 2^N possible runtime states, most of which are never tested and some of which are logically impossible (flag B assumes flag A's new-and-improved code path exists, but flag A is still off in some environment).
- **Dead flags:** a flag fully rolled out to 100% six months ago, still gated by an `if`, still imposing a runtime lookup and a "what does the false branch even do anymore" question for every new engineer reading the code.
- **Config vs. code drift:** the flag's default in the flag-management UI says one thing, the code's fallback-on-load-failure says another, and nobody has verified whether they still agree.
- **Testing burden:** QA either tests every flag combination (impossible) or only the "current" combination, quietly making all other combinations untested and effectively broken without anyone knowing.

Mitigations: treat *release* flags as inherently short-lived and put an explicit expiry/owner on each one at creation time (many flag platforms support this natively); a recurring "flag cleanup" backlog ritual or automated linter that flags (pun intended) any flag whose rollout has been at 100%/0% for N weeks as a candidate for removal; and architecturally, keep flag checks at the boundary (e.g., which component/route to render, per 3.2) rather than sprinkled through business logic, so removing a flag later is a small, mechanical diff instead of an archaeology project.

6. Interview Questions & Answers

Q1. Why would you choose vertical (domain-driven) folder slicing over grouping by technical layer (components/, services/, models/) in a large app?

Technical layering creates a small number of very hot folders that every team touches, causing constant merge conflicts and making ownership impossible to establish. Domain/vertical slicing groups all the code for one business capability (models, services, components) together, so a domain maps to a team, a PR touches one slice, and cross-team merge conflicts drop sharply. It also makes deletion/extraction of a whole feature trivial — delete the folder — versus technical layering where a feature's code is scattered across five top-level folders.

Q2. What are Nx's `type` and `scope` tags for, and how do they interact with `@nx/enforce-module-boundaries`?

`type` (feature/data-access/ui/util) encodes an architectural layering rule — features can use anything, util should depend on nothing app-specific — forming a DAG that keeps business logic out of presentational components and keeps low-level utilities dependency-free. `scope` (orders/billing/shared) encodes domain ownership — a domain lib may depend on itself or on `shared`, never sideways into another domain.

`@nx/enforce-module-boundaries` reads both sets of tags off each project and statically checks every import against configured `depConstraints`, failing lint if a project imports from another project whose tags aren't in its allowed list. This converts an architecture diagram into an enforced, CI-blocking rule instead of documentation nobody reads.

Q3. How does `nx affected` decide what to test/build/lint, and why is this faster than running everything?

Interviewer intent: checks whether the candidate understands Nx isn't magic — it's git diffing plus a dependency graph plus content hashing, and can explain *why* it's correct, not just that it's fast.

Nx diffs the changed files between the current branch and a base ref, maps each changed file to the project that owns it, then walks the project graph to include every project that transitively depends on any directly-changed project (since a change in a dependency can ripple to dependents). Only that "affected" set runs its tasks. It's faster than running everything because in a typical PR touching a handful of files, the affected set is a small fraction of a 300-project workspace — and on top of that, Nx's task hashing (source files + transitive dependency source files + config + env) means even within the affected set, if a task's hash matches something cached (locally or via Nx Cloud), Nx replays the cached result instead of re-executing, so CI time scales with the size of the actual change, not the size of the repo.

Q4. Your team wants to publish the shared design system as a versioned npm package instead of keeping it as a path-mapped Nx lib. What do you gain, and what new problems does this introduce?

You gain the ability for teams outside the monorepo (or on separate release cadences) to consume the design system, and you decouple its release train from the rest of the workspace. You introduce version-skew risk — different consumers on different major versions, seeing visually inconsistent UI — plus diamond-dependency conflicts if two consumed packages depend on incompatible versions of a shared transitive dependency, plus real coordination cost for breaking changes (deprecation windows, migration schematics, multi-team rollout scheduling) that a monorepo would have resolved in a single atomic commit. In practice, most orgs keep fast-moving domain libs path-mapped inside the monorepo and only publish the handful of libraries that genuinely need cross-repo/cross-org consumption.

Q5. Explain the difference between a build-time `environment.ts` file replacement and a runtime configuration approach. Why does it matter for enterprise environments?

Interviewer intent: looking for whether the candidate has actually shipped something through a real dev→staging→prod pipeline, versus only having used the CLI's default `environment.prod.ts` swap for toy apps.

`environment.ts` / `environment.prod.ts` file replacement bakes values into the compiled JS bundle at build time — different environments require different builds, meaning the artifact tested in staging is literally not the same bytes deployed to prod, undermining staging as a genuine pre-prod gate. Runtime configuration fetches a small JSON (or calls a config endpoint) via an `APP_INITIALIZER` / `provideAppInitializer` before the app renders, and only that JSON differs per environment — the JS/CSS bundle is identical across dev/staging/prod. This gives "build once, promote everywhere" — the actual CI/CD ideal — and lets ops change an API URL or feature-flag endpoint by swapping a config file, with no rebuild, no redeploy of the app bundle, and no risk of environment-specific compiled code diverging from what was tested.

Q6. What's the API-layer abstraction problem this is solving, and where exactly should the DTO-to-domain-model mapping live?

Without an abstraction layer, components call `HttpClient` directly and consume raw backend DTOs, so any backend contract change (renamed field, restructured payload, v1→v2 migration) forces a shotgun-surgery refactor across every component that touched that data shape. The mapping should live in a single `data-access` / repository layer per domain — the only code that knows the wire format — which converts DTOs into stable internal domain models and exposes use-case-shaped methods through a facade/store. Components and feature libs depend only on the domain model and the facade's methods, never on the DTO shape, so a backend change is a one-file diff in the repository layer instead of a codebase-wide hunt.

Q7. How would you support a backend v1→v2 API migration without touching every component that consumes that data?

Keep the migration entirely inside the domain's repository/API layer: implement a v2-aware fetch alongside the existing v1 fetch, map both to the *same* internal domain model shape, and put the version selection behind a flag or a gradual server-side rollout inside the repository (or behind the feature-flag service from 2.7/3.2 if it needs to be toggled per user/tenant). The facade/store above it and every component above that never change, because they only ever depended on the stable domain model, not on the DTO. Once v2 is fully rolled out, the v1 code path in the repository is deleted — a small, contained diff.

Q8. Two teams keep colliding on a "shared utility" that keeps changing shape and breaking each other's code. How do you architecturally prevent this?

This is almost always a scope-boundary problem: the "shared utility" is actually a piece of one domain's model leaking into general use, or it needs to be pulled into a genuinely stable, low-churn `scope:shared`, `type:util` lib with an explicit ownership team and a stricter change-review bar than domain code gets (since util libs sit at the bottom of the dependency DAG and everything depends on them). Concretely: extract it into its own lib, tag it `scope:shared`, assign a platform/owning team, require the util lib's public API to be additive-only or explicitly versioned/deprecated rather than freely reshaped, and let `@nx/enforce-module-boundaries` plus code ownership (CODEOWNERS) enforce that domain teams can consume it but not casually redefine its contract.

Q9. What's a circular dependency between two Nx libraries, why does it break `affected`, and how do you detect and fix it?

A cycle exists when lib A depends (directly or transitively) on lib B and lib B depends back on A — e.g., a `util` lib importing a helper from a `feature` lib "just this once." This breaks the DAG assumption `affected`'s change-propagation and any topological build-ordering rely on: within a cycle, there's no valid build order, and

Nx either errors outright with cycle detection enabled, or, undetected, causes every change anywhere in the cycle to mark the *entire* cycle as affected indefinitely, destroying the fine-grained skip-what-didn't-change benefit of `affected`. Detect it by enabling the circular-dependency check in `@nx/enforce-module-boundaries` (or visualizing with `nx graph` / `dependency-cruiser`'s graph output) so it fails CI immediately. Fix it by extracting the mutually-needed piece into a new, lower-layer shared lib that both original libs depend on downward, breaking the cycle into a proper DAG.

Q10. When is splitting code into more, smaller Nx libraries actually harmful, and what's the corrective principle?

Interviewer intent: tests whether the candidate parrots "always split into small libs" (outdated cargo-culting of early Nx advice) or actually understands the tradeoff.

Excess fragmentation adds fixed per-project overhead (project.json/config parsing, executor startup, barrel-file/public-API boilerplate for libs nobody consumes independently) and makes the codebase harder to navigate — understanding one feature now means opening a dozen near-empty libs instead of one cohesive folder — without providing real isolation, because nothing outside that feature ever consumed the split-out piece independently anyway. The corrective principle is to split at genuine reuse or genuine independent-ownership boundaries: extract a new lib when a second consumer needs only part of an existing lib, or when a different team needs to own/version/test that piece independently — not preemptively, and not just because a file got long.

Q11. How do ESLint custom rules and `dependency-cruiser` complement `@nx/enforce-module-boundaries`, and when would you reach for one over the other?

`@nx/enforce-module-boundaries` is specifically about the Nx project graph's type/scope tag constraints and is Nx-aware (understands `project.json`, tags, buildable-lib constraints). Custom ESLint rules (via `@typescript-eslint/utils`) enforce org-specific AST-level conventions that aren't about project boundaries at all — banning direct `HttpClient` injection outside `data-access` libs, requiring a component selector prefix, banning `any` in feature code — things expressible as "does this specific syntax pattern appear in this file," which the Nx rule doesn't cover. `dependency-cruiser` is a general, Nx-independent dependency-graph linter useful in plain Angular workspaces without Nx, or for cross-checking dependency rules the Nx rule doesn't express well (e.g., forbidding `core` from importing `features`, or generating a visual `.svg` dependency graph for architecture reviews). In practice all three run together in CI: Nx's rule for the type/scope DAG, custom ESLint for org conventions, `dependency-cruiser` as a second, tool-independent check and for visualization.

Q12. Describe the lifecycle of a feature flag and what "feature flag debt" looks like in a codebase, and how you'd prevent it.

A release flag's lifecycle should be: created with an explicit owner and expected rollout timeline → gates the new code path in a small number of boundary locations (route/component selection, not scattered through business logic) → progressively rolled out (1%→10%→100%) → once stable at 100% for a defined window, the flag and its old code path are deleted in a follow-up PR. Debt accumulates when that last step never happens: flags stuck at 100% or 0% for months, still evaluated at runtime, still bifurcating code paths that are no longer really "in flux," creating up to 2^N untested state combinations, and config-vs-code drift where the flag platform's stated default disagrees with the code's fail-safe fallback. Prevention: track an expiry/owner on every flag at creation, run a recurring cleanup ritual (or automated lint/report on long-stable flags), and keep flag checks confined to a small number of boundary decision points so removing a flag later is a mechanical, contained diff rather than an archaeology project through business logic.

Q13. Walk through what happens end-to-end when a developer opens a PR that adds one new field to `orders-data-access` and imports it from `orders-feature-order-list` — from local lint through CI.

Interviewer intent: wants to see the candidate connect tags → lint → project graph → affected → caching into one coherent pipeline, not just recite definitions in isolation.

Locally, on save, ESLint (via the Nx flat config) parses the changed file's imports; since `orders-feature-order-list` (`type:feature`, `scope:orders`) importing `orders-data-access` (`type:data-access`, `scope:orders`) satisfies both the type DAG (feature may depend on data-access) and the scope rule (same scope), `@nx/enforce-module-boundaries` passes. On push, CI runs `nx affected -t lint,test,build` against the PR's base ref; Nx diffs changed files, maps the changed file in `orders-data-access` to that project, and walks the graph to find every project depending on it transitively — including `orders-feature-order-list` and any app that bundles it — marking that whole set affected while leaving `billing-*` and unrelated `shared-*` libs (assuming they don't depend on `orders-data-access`) out of scope entirely. For each affected task, Nx computes a content hash from the project's source plus its transitive dependencies' source plus config; any task whose hash doesn't match a previous cache entry actually executes, and results are cached (locally and/or via Nx Cloud) for the next run. Assuming lint/test/build succeed, the PR merges; the next deploy pipeline promotes the same built artifact through staging and production, configured only by the runtime `config.json`, never rebuilt per environment.

Q14. If you inherited a large Angular monorepo with none of these guardrails — no tags, no lint boundary enforcement, a tangle of cross-imports — how would you introduce this architecture incrementally without freezing feature development?

Start by generating a dependency graph (`nx graph` or `dependency-cruiser` 's visual output) to see the actual current state, including existing cycles, before assuming a clean slate. Introduce tags on the highest-value/most-actively-developed libs first, add `@nx/enforce-module-boundaries` in **warn** mode (not error) so existing violations surface without blocking every in-flight PR, then triage: genuine architecture violations get fixed or explicitly grandfathered via `allow` lists with a tracked follow-up ticket, and only once the warning count is near zero do you flip the rule to **error** for that constraint. Do this constraint-by-constraint (e.g., enforce `type` boundaries before `scope` boundaries, or vice versa depending on which is more violated) rather than all at once, so teams absorb one dimension of enforcement at a time. In parallel, break obvious cycles (5.1) since those block `affected` from ever being reliable, and only after boundaries are stable and enforced does introducing feature flags, environment/config abstraction, and library versioning policy become worth the investment — sequencing structural/dependency sanity before process-level guardrails avoids fighting a moving, tangled target.

7. Quick Revision Cheat Sheet

- **Vertical slicing > technical layering:** group by business domain (orders/billing), not by type (components/services), so teams own folders, not everyone touching the same hot files.
- **Nx tags** = `type` (feature/data-access/ui/util — an intra-domain DAG) × `scope` (orders/billing/shared — a domain ownership boundary). Both enforced together by `@nx/enforce-module-boundaries`.
- **affected mechanism:** git diff → changed files → owning projects → walk graph for transitive dependents → run tasks only for that set; task hashing (source + transitive deps + config) drives cache hits/skips.
- **DAG integrity is load-bearing:** a circular lib dependency breaks both correct build ordering and the precision of `affected` — detect via cycle-detection lint rule or `nx graph / dependency-cruiser`.
- **Design system:** components + design tokens + Storybook contract + visual regression tests; path-mapped in-repo avoids version skew, published npm package trades that for cross-repo reach.

- **Library versioning:** path-mapped/in-monorepo = no version skew, single repo required; published/semver = cross-repo reach, but version skew + diamond deps + deprecation coordination cost.
- **Feature flags:** abstract behind a `FeatureFlagService`, resolve before render via `APP_INITIALIZER / provideAppInitializer`, distinguish short-lived release flags (must expire) from long-lived kill-switch/entitlement flags.
- **Environment config:** build once, configure at deploy time via a fetched runtime `config.json` + `ConfigService`, not `environment.prod.ts` file replacement (which bakes values into the bundle and breaks "same artifact through every stage").
- **Enforcement stack:** `@nx/enforce-module-boundaries` (Nx-aware type/scope DAG) + custom ESLint rules (org conventions, AST-level) + `dependency-cruiser` (tool-agnostic graph checks/visualization) — all run in CI as blocking gates, scoped by `affected`.
- **API layer abstraction:** repository/data-access layer owns DTO↔domain-model mapping; facades expose use-case methods and domain models; components never see raw wire format — backend contract changes become one-file diffs.
- **Common failure modes:** circular lib deps, over-fragmented tiny libs, shared-lib version skew across teams, and feature-flag debt (dead flags, untested combinatorics, config/code drift) — each has a specific structural fix, not just "be more careful."

Created By - Durgesh Singh